

Análisis y Diseño con Patrones

PEC 2: Patrones de Diseño y de Asignación de Responsabilidades

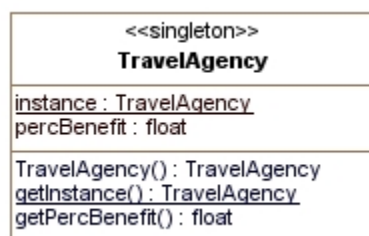
Pregunta 1 (25%)

Suponed que hemos diseñado un sistema software para una única agencia de viajes. Las clases Viaje y Cliente registran toda la información de los viajes que contratan los clientes a la agencia. La agencia quiere registrar información propia de la agencia como, por ejemplo, el % de beneficio que aplica al precio de los viajes (es decir, el % que quiere ganar la agencia de cada viaje contratado). Este % de beneficio es el mismo para todos los viajes. Se pide:

- ¿Qué patrón de diseño es adecuado para grabar este % de beneficio?
- Haz el diagrama de clases resultante de aplicar este patrón (solo hace falta la parte de aplicación del patrón).
- Escribe el pseudocódigo del diagrama de clases que propones en el apartado anterior. Solo hace falta la parte de aplicación del patrón.

Solución

- Aplicamos el patrón *Singleton* en una nueva clase *Agencia* para mantener una única instancia de esta clase que tenga el % de beneficio que quiere ganar la agencia, de modo que esta información esté registrada una única vez en la agencia.
- El diagrama de clases resultante del patrón tendrá definida una única clase.



- ```

public class TravelAgency {
 private static TravelAgency instance = null;

```

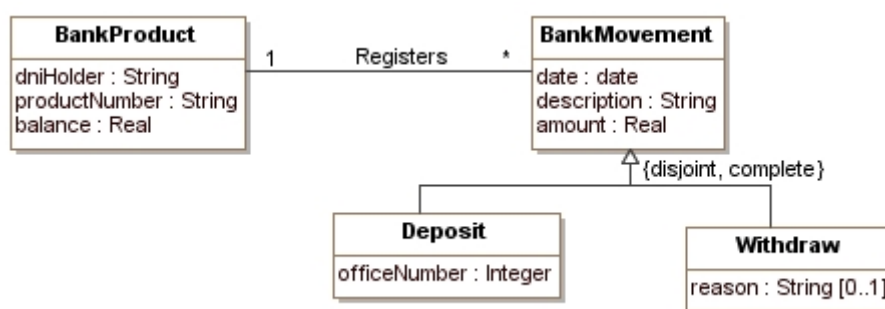
```

private float percBenefit;
private TravelAgency() {}
public static TravelAgency getInstance() {
 if (instance == null) {
 instance = new TravelAgency();
 }
 return TravelAgency;
}
public float getPercBenefit {
 return percBenefit;
}
}

```

## Pregunta 2 (25%)

Suponed que queremos diseñar un sistema software para una entidad bancaria. El sistema registra los productos bancarios que se identifican por su número y los movimientos (ingresos y gastos) que se hacen en estos productos bancarios. Queremos diseñar la operación *deposit* (*quantity: Real*, *officeNumber: Integer*) de la clase *BankProduct*. A continuación, disponéis de un fragmento del diagrama de clases del sistema:



La operación que queremos diseñar crea un ingreso con todos los datos, lo asocia con el producto bancario y actualiza el saldo de acuerdo con el importe del ingreso. Esta operación tiene la precondición (condición que se satisface antes de invocar la operación):

- El parámetro *amount* es mayor que 0.

La operación tiene que hacer lo siguiente:

- Crear un objeto *Deposit dep*, asignarle el importe *quantity*, la fecha de hoy (podéis invocar la operación *getDate():date* desde donde la necesitáis para obtener esta fecha), la descripción “DEPOSIT”, y el número de oficina *officeNumber*.
- Registrar al producto bancario el objeto *dep* creado.
- Incrementar el *balance* del producto bancario con el importe del ingreso.

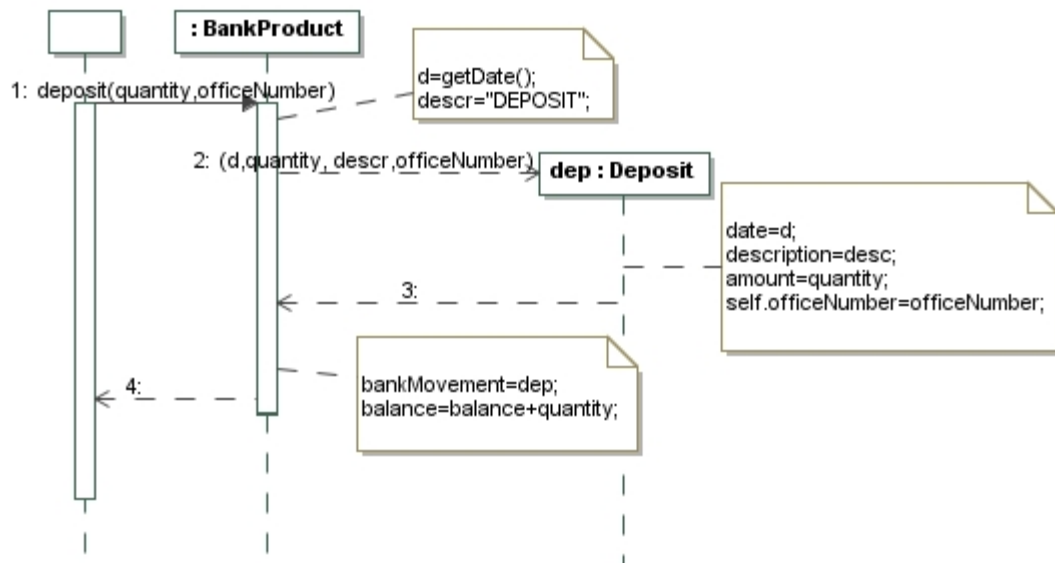
Se pide:

- a) ¿A qué clase corresponde la responsabilidad de incrementar el saldo del producto bancario cuando se produce un ingreso? Justifica la respuesta.
- b) ¿Qué patrones de diseño son adecuados aplicar a la operación *deposit*?
- c) Escribe el pseudocódigo o el diagrama de secuencia de la operación *ingreso* y de todas las operaciones que sean invocadas desde esta operación.

Nota: Para simplificar, hemos definido las cantidades que se utilizan en este ejercicio como Real, a pesar de que tendríamos que haber utilizado el patrón Quantity.

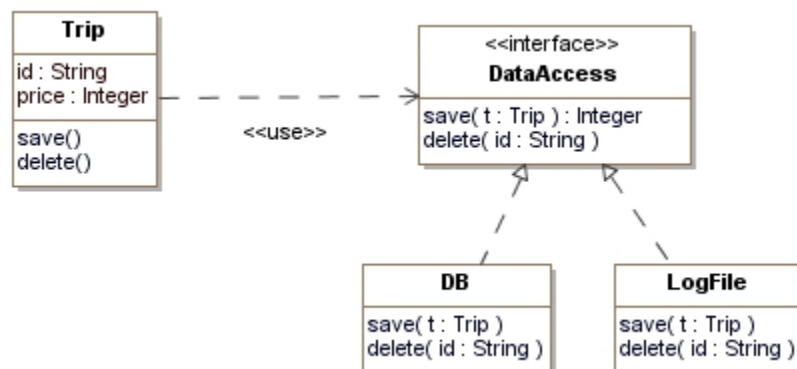
## Solución

- a) La clase *BankProduct* será la responsable de incrementar el saldo cuando se produzca un ingreso puesto que es la clase que tiene el atributo saldo.
- b) En la operación *deposit* se han aplicado el patrón Experto para asignar las responsabilidades a las dos clases que participan en la operación y el patrón Creador para hacer la creación del objeto *deposit*.
- c) A continuación disponéis del diagrama de secuencia de la operación:

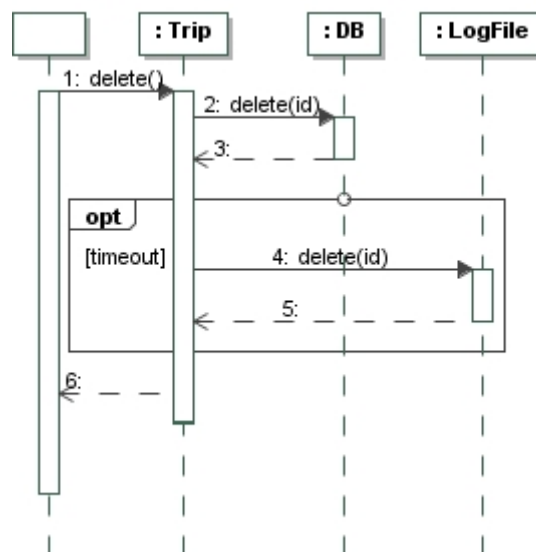


## Pregunta 3 (25%)

La agencia de viajes de la Pregunta 1 almacena los viajes en una base de datos. En algunas ocasiones, cuando se quieren guardar o borrar los viajes, la base de datos no está disponible (las operaciones de guardar y borrar devuelven una excepción de timeout). Como es importante para la agencia garantizar que la información quede registrada en la base de datos, hemos decidido diseñar un sistema de backup para solventar este problema. Este backup consistirá en un fichero log donde, si no se puede registrar la información en la base de datos, se registrará en este log. Existe un proceso (que nosotros no tenemos que diseñar) que se encarga de trasladar la información del log a la base de datos cuando esta esté disponible. A continuación disponéis del diagrama de clases de esta solución.



Nos damos cuenta que esta solución es mejorable puesto que la gestión de la invocación en la base de datos o al log (en caso de no disponibilidad de la base de datos) lo tienen que hacer las operaciones de la clase Viaje. Esto supone un problema puesto que la clase Viaje tiene que interactuar directamente con las dos clases (aumentando el acoplamiento) y, además, esta gestión se tiene que repetir en cada una de las operaciones. A continuación disponéis del diagrama de secuencia de la operación *delete()* de la clase *Trip*.



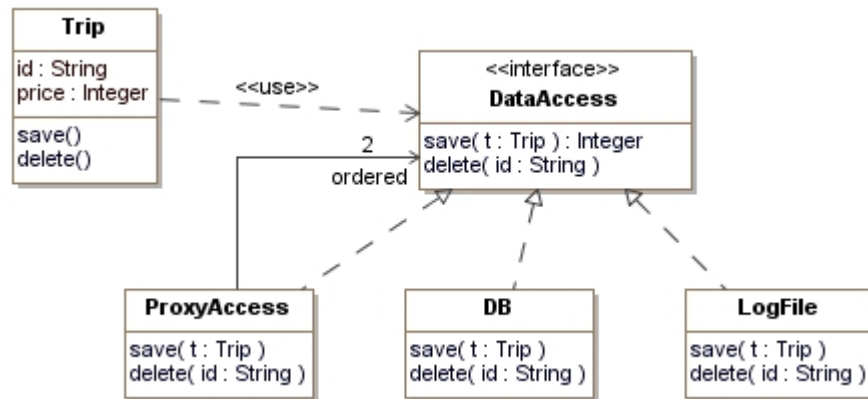
Se pide:

- ¿Qué patrón de diseño recomendarías para mejorar esta solución (si es que recomiendas alguno)?
- Muestra la parte del diagrama de clases que se ve modificada como resultado de la aplicación del patrón utilizado.
- Muestra el diagrama de secuencia de la operación *delete()* de la clase *Trip*.

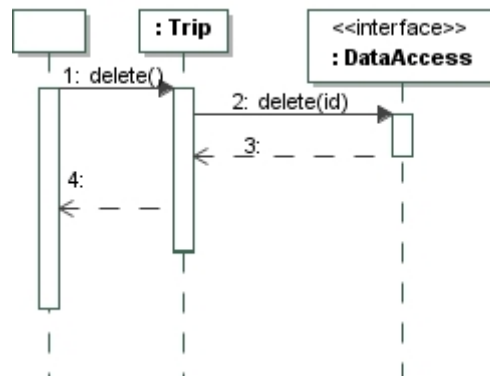
Nota: Si hace falta, podéis modificar ligeramente el patrón presentado en los materiales.

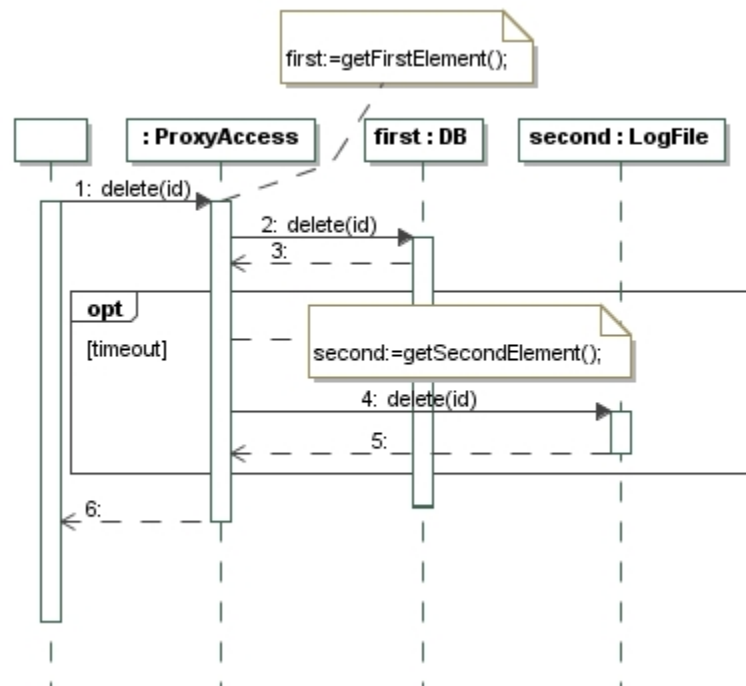
## Solución

- Para mejorar el acoplamiento de nuestra solución y controlar el acceso a las clases *DB* y *LogFile* aplicaremos el patrón *Representante (Proxy)*.
- El diagrama de clases muestra la aplicación del patrón *Representante* añadiendo una nueva clase *ProxyAccess* que implementa la interfaz *DataAccess*. Esta clase tiene la referencia a las dos clases *DB* y *LogFile* y gestiona a quién tiene que invocar.



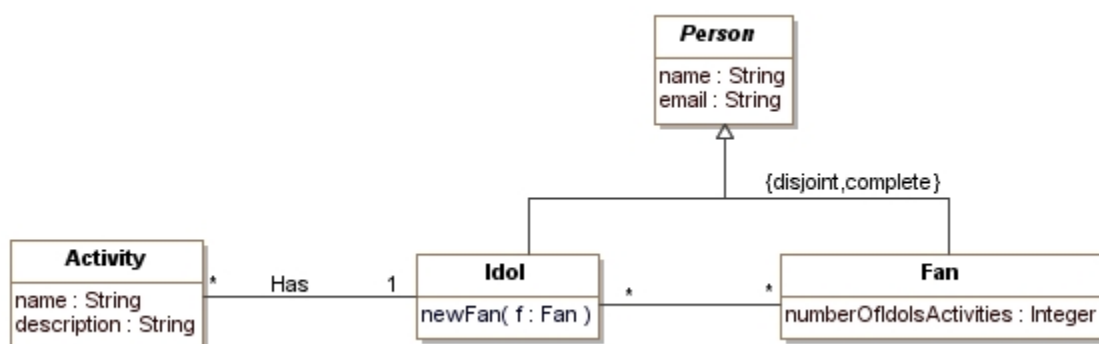
- c) El nuevo diagrama de secuencia de la operación `delete()` de la clase *Trip* se muestra a continuación:





## Pregunta 4 (25%)

Suponed que estáis diseñando un sistema que registra las personas existentes en una comunidad. De las personas registramos su nombre y email. Las personas pueden ser ídolos o fans. De los ídolos sabemos la descripción de su última actividad y los fans que tienen. De los fans sabemos sus ídolos y el número total de fans que comparten alguno o más ídolos. De momento, tenéis el diagrama de clases siguiente:



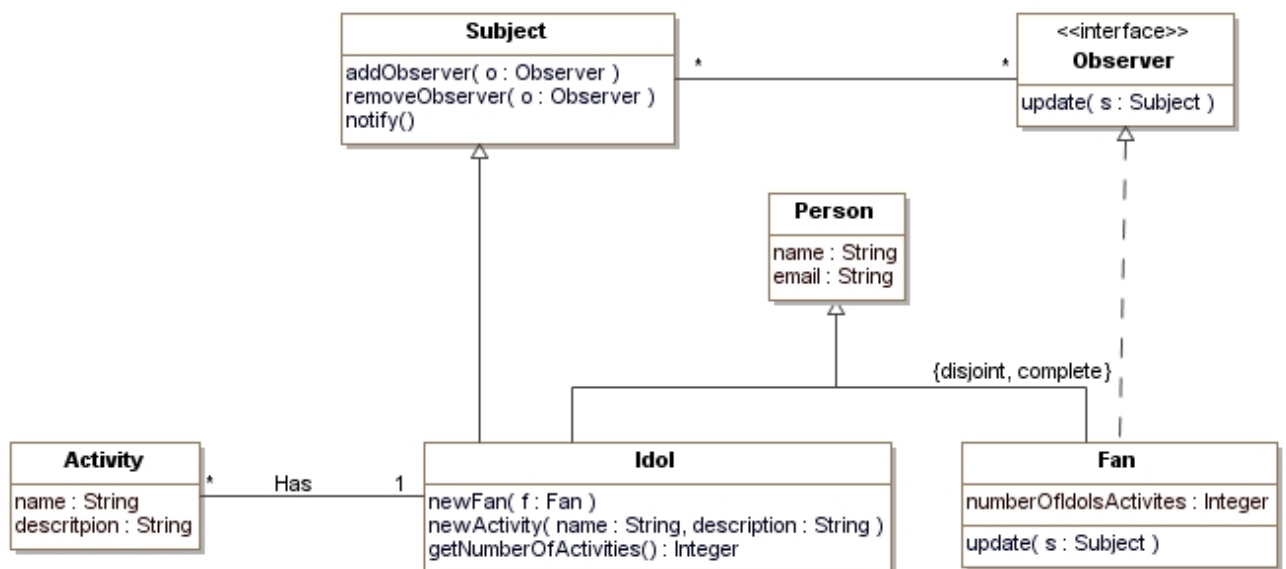
Queremos diseñar la operación `newActivity(name: String, description:String)` de la clase `Idol`. Esta operación permite dar de alta una nueva actividad del ídolo, asociarla con él y, como consecuencia,

modificar el *numberOfIdolsActivities* de sus fans. Para diseñar esta operación se quiere evitar al máximo el acoplamiento entre los ídolos y sus fans. Se pide:

- ¿Qué patrón de diseño recomendarías para este caso (si es que recomiendas alguno)?
- Muestra el diagrama de clases resultante de añadir esta funcionalidad.
- Muestra el diagrama de secuencia o el pseudocódigo de la operación *newActivity* de la clase *Idol*.

## Solución

- Aplicamos el patrón *Observador*, puesto que nos permite evitar el acoplamiento entre ídolo y fan cuando se produce un cambio en ídolo (la creación de la nueva actividad y de la asociación). Cuando se produzca una nueva actividad de ídolo, la operación notificar del patrón se encargará de actualizar todos los observadores (los fans del ídolo) que estén asociados. La operación de actualizar será la encargada de modificar el atributo *numberOfIdolsActivities*.
- A continuación, disponéis del diagrama de clases donde hemos aplicado el patrón *Observador*:



- A continuación, se muestra el diagrama de secuencia de la operación *newActivity(description:String)*.



